Problem ID	Captured Flag	Steps (Summary)
P1 – dossier_101	`fsuCTF{p1ca55o_w0u1dv3_l0v3d_m5pa1n7}`	Mount VHD with qemu-nbd, explore NTFS filesystem, locate masterpiece.bmp in Paint app data, view image to reveal flag
P2 – Jaws	`fsuCTF{Y0ur3_G0nn4_N33d_A_B1g3r_Bo4t}`	Inspect PCAP protocol hierarchy, filter HTTP traffic to suspicious IP, extract and base64-decode URI parameters to recover flag
P3 – What's the Title Again?	`fsuCTF{1_d0n7_w4n7_t0_pl4y_4nym0r3}`	Run strings on memory dump, grep for mp3/media references, find wmplayer.exe command with Sleep Away.mp3, locate in-memory title string

P1 – dossier_101

This challenge provided a VHD disk image (xp_sm01.vhd) of a Windows XP machine. The goal was to mount the disk, explore the filesystem, and locate a hidden flag embedded in an image file.

Mounting the Disk

The VHD was connected using qemu-nbd and then mounted read-only to inspect the NTFS partition:

```
$ sudo qemu-nbd --connect=/dev/nbd0 xp_sm01.vhd
$ sudo fdisk -l /dev/nbd0
  -> NTFS partition at /dev/nbd0p1

$ sudo mkdir -p /mnt/xp
$ sudo mount -o ro /dev/nbd0p1 /mnt/xp
```

Investigation

1. Navigated to the user directory: /mnt/xp/Documents and Settings/daniel
2. Discovered an RTF file referencing a saved 'masterpiece' image

3. Located the image using find:

```
$ find /mnt/xp -iname "masterpiece*"
/mnt/xp/Documents and Settings/daniel/Application
Data/Microsoft/Paint/masterpiece.bmp
```

4. Opened the image with xdg-open and the flag was visible in the image content

```
$ xdg-open masterpiece.bmp
```

Flag: **fsuCTF{p1ca55o_w0u1dv3_l0v3d_m5pa1n7}**

P2 – Jaws

This challenge provided a packet capture file (jaws.pcap). The goal was to find evidence of data exfiltration by analyzing the network traffic.

Protocol Analysis

First inspected the protocol hierarchy to get an overview of the traffic:

```
$ tshark -r jaws.pcap -q -z io,phs
```

Observed HTTP traffic and suspicious repeated requests to a specific IP address (178.63.67.153). Filtered to isolate those requests:

```
$ tshark -r jaws.pcap -Y "ip.dst == 178.63.67.153 && http.request"
```

Extracting & Decoding the Flag

1. Identified repeated GET requests with encoded query parameters in the URI
2. Extracted the URI parameters:

```
$ tshark -r jaws.pcap -Y "ip.dst == 178.63.67.153 && http.request" \
    -T fields -e http.request.uri
```

3. The URI contained base64-encoded data split across multiple requests
4. Cleaned URL encoding, restored padding, and decoded the reassembled base64 string using Python
5. Decoded output revealed the flag

Flag: **fsuCTF{Y0ur3_G0nn4_N33d_A_B1g3r_Bo4t}**

P3 – What's the Title Again?

This challenge provided multiple memory dump files: memory.vmem, what.raw, and whats_the_title_again.raw. Using the file command, the .raw files were identified as ELF 64-bit core dumps. The goal was to identify what media was playing at the time of the dump.

Searching for Media References

Searched for mp3-related strings in the target dump using wide-character string extraction:

```
$ strings -a -el -n 4 whats_the_title_again.raw | grep -i mp3
```

Observed repeated references to Windows Media Player and sample music files. Narrowed down to active playback:

```
$ strings -a -el whats_the_title_again.raw | grep -i "/Play"

wmplayer.exe /Relaunch /Play "C:\Users\Public\Music\Sample Music\Sleep Away.mp3"
```

Recovering the Flag

1. Identified wmplayer.exe was actively playing Sleep Away.mp3 at time of dump
2. Continued searching memory around the playback context for the in-memory title string
3. Discovered the flag string directly in memory: 1_d0n7_w4n7_t0_pl4y_4nym0r3

Flag: **fsuCTF{1_d0n7_w4n7_t0_pl4y_4nym0r3}**